

Week 5 Reflection

Anomaly Detection

I used Isolation Forest to detect anomalies in CPU utilization, error rate, and p99 latency. I tested contamination values of 0.01, 0.04, and 0.10.

A contamination value of 0.04 produced the best overall result, with precision of 0.80, recall of 1.00, and F1 score of 0.89. It detected all 16 true anomalies but also produced four false positives.

The experiment showed that a lower contamination value reduced false positives but missed real anomalies, while a higher value improved recall but created too many false alarms.

Alert Grouping and RCA

I grouped alerts using a five-minute time-window rule. This reduced multiple related anomaly alerts into a smaller number of incidents.

The RCA agent used two tools: one for retrieving metric evidence and one for retrieving logs. The main incident showed high CPU usage, a high error rate, p99 latency above 3.6 seconds, and multiple database connection errors.

The most likely root cause was database connection pool exhaustion.

Observability and Cost

I instrumented the RCA agent with OpenTelemetry and captured GenAI attributes including model name, input tokens, output tokens, latency, and estimated cost.

The simulated experiment showed that input token usage increased approximately linearly as log window size increased. Because no real LLM API was used, the actual API cost was zero.

A production system should include guardrails such as maximum log-window size, token limits, request-rate limits, and cost budgets to prevent runaway LLM spending.

What Did Not Work as Expected

One issue I encountered was that the exported span file was initially written as JSON Lines instead of a valid JSON array. VS Code reported a JSON formatting error. I fixed the issue by converting the captured span output into a valid JSON array.

This reinforced the importance of validating telemetry output formats instead of assuming that successful span generation automatically means the output is ready for downstream tools.